Examen Sistemas expertos

Nombre:

Andrés Zeas G.

Objetivo

Consolidar los conocimientos adquiridos en clase de los sistemas expertos basados en reglas.

Enunciado

Se desea modelizar el conocimiento de un experto para el diagnóstico de enfermedades de las plantas, basadas en las siguientes reglas.
• Las plantas requieren diferentes tipos de nutrientes para desarrollarse adecuadamente.

• Tres de los nutrientes de mayor importancia son el nitrógeno, el fósforo y el potasio.

• Una deficiencia en alguno de estos nutrientes puede producir varios síntomas.

Clips + Python

Para instalar la libreria solo tenemos que ejecutar

`pip install clipspy`

Procedemos a verificar que la instalacion este correcta con la siguiente instrucción.

In [8]:
```
from clips import Environment, Symbol

environment = Environment()
```

Procedemos a cargar nuestro sistema basado en reglas, para ello utilizamos el metodo load y le enviamos el archivo .clp

In [9]:
```
environment.load('Plantas.clp')
```

Ahora validamos que se cargo el archivo de reglas con la siguiente instrucción.

In [10]:
```
environment.reset()
environment.run()
```

Out[10]:
```
0
```

Podemos mostrar la base de reglas y los hechos con las siguentes instrucciones.

In [11]:
```
for rule in environment.rules():
    print(rule)

for fact in environment.facts():
    print(fact)
```

```
(defrule MAIN::pregunta1
   (respuesta1 si)
   =>
   (assert (planta-crece-poco)))

(defrule MAIN::pregunta2
   (respuesta2 si)
   =>
   (assert (planta-color-amarillo-palido)))

(defrule MAIN::pregunta3
   (respuesta3 si)
   =>
   (assert (hojas-color-pardo-rojizo)))

(defrule MAIN::pregunta4
   (respuesta4 si)
   =>
   (assert (raiz-crece-poco)))

(defrule MAIN::pregunta5
   (respuesta5 si)
   =>
   (assert (planta-tallo-fusiforme)))

(defrule MAIN::pregunta6
   (respuesta6 si)
   =>
   (assert (planta-color-purpura)))

(defrule MAIN::pregunta7
   (respuesta7 si)
   =>
   (assert (planta-retraso-madurez)))

(defrule MAIN::respuesta8
   (respuesta8 si)
   =>
   (assert (bordes-hojas-chamuscados)))

(defrule MAIN::pregunta9
   (respuesta9 si)
   =>
   (assert (planta-tallos-debilitados)))

(defrule MAIN::pregunta10
   (respuesta10 si)
   =>
   (assert (semillas-frutas-marchitas)))

(defrule MAIN::ausencia-nitrogeno
   (or  (planta-crece-poco)
        (planta-color-amarillo)
        (hojas-color-pardo-rojizo))
   =>
   (assert (no-nitrogeno))
   (printout t "La planta carece de Nitrógeno." crlf))

(defrule MAIN::ausencia-fosforo
   (or  (raiz-crece-poco)
        (planta-tallo-fusiforme)
        (planta-color-purpura)
        (planta-retraso-madurez))
   =>
   (assert (no-fosforo))
   (printout t "La planta carece de Fósforo." crlf))

(defrule MAIN::ausencia-potasio
   (or  (bordes-hojas-chamuscados)
        (planta-tallos-debilitados)
        (semillas-frutas-marchitas))
   =>
   (assert (no-potasio))
   (printout t "La planta carece de Potasio." crlf))

(initial-fact)
```

Desarrollo

Una vez comprobado que todo esta funcionando correctamente procedemos a realizar la interacción con el usuario mediante un sistema de encuenta el cual va a preguntar si la planta presenta o no un determinado sintoma a lo que el usurio debera responder con un `si` o un `no`
Lo que logramos con esto es crear hechos en nuestra base, los mismos que van a determinar cuales son los nutrientes que

In [12]:
```
respuesta1= input('La planta crece muy poco? (si/no) \n')
print(respuesta1)
environment.assert_string('(respuesta1 '+respuesta1+')')

respuesta2= input('La planta tiene un color amarillo pálido. (si/no) \n')
print(respuesta2)
environment.assert_string('(respuesta2 '+respuesta2+')')

respuesta3= input('Las hojas tienen un color pardo rojizo? (si/no) \n')
print(respuesta3)
environment.assert_string('(respuesta3 '+respuesta3+')')

respuesta4= input('La raiz de la planta tiene poco crecimiento? (si/no) \n')
print(respuesta4)
environment.assert_string('(respuesta4 '+respuesta4+')')

respuesta5= input('Una planta con tallo fusiforme? (si/no) \n')
print(respuesta5)
environment.assert_string('(respuesta5 '+respuesta5+')')

respuesta6= input('Una planta con color púrpura? (si/no) \n')
print(respuesta6)
environment.assert_string('(respuesta6 '+respuesta6+')')

respuesta7= input('Un retraso en la madurez de una planta? (si/no) \n')
print(respuesta7)
environment.assert_string('(respuesta7 '+respuesta7+')')

respuesta8= input('Los bordes de las hojas aparecen chamuscados? (si/no) \n')
print(respuesta8)
environment.assert_string('(respuesta8 '+respuesta8+')')

respuesta9= input('Una planta con los tallos debilitados? (si/no) \n')
print(respuesta9)
environment.assert_string('(respuesta9 '+respuesta9+')')

respuesta10= input('Una planta con semillas o frutas marchitas? (si/no) \n')
print(respuesta10)
environment.assert_string('(respuesta10 '+respuesta10+')')
```

```
La planta crece muy poco? (si/no)
no
no
La planta tiene un color amarillo pálido. (si/no)
no
no
Las hojas tienen un color pardo rojizo? (si/no)
no
no
La raiz de la planta tiene poco crecimiento? (si/no)
si
si
Una planta con tallo fusiforme? (si/no)
no
no
Una planta con color púrpura? (si/no)
si
si
Un retraso en la madurez de una planta? (si/no)
no
no
Los bordes de las hojas aparecen chamuscados? (si/no)
no
no
Una planta con los tallos debilitados? (si/no)
si
si
Una planta con semillas o frutas marchitas? (si/no)
no
no
```

Out[12]:
```
ImpliedFact: f-10    (respuesta10 no)
```

Una vez terminada la encuentra mandamos a ejecutar nuevamente la base para que se actualicen las reglas y nos determine cuales son las carencias de nutrientes.

In [14]:
```
environment.run()

for fact in environment.facts():
    if fact.template.name == 'no-nitrogeno':
        print('La planta carece de Nitrógeno')
    elif fact.template.name == 'no-fosforo':
        print('La planta carece de Fósforo')
    elif fact.template.name == 'no-potasio':
        print('La planta carece de Potasio')
```

```
La planta carece de Potasio
La planta carece de Fósforo
```

Podemos nuevamente listar los hechos y ver que efectivamente estan cargados.

In [10]:
```
for fact in environment.facts():
    print(fact)
```

```
(initial-fact)
(respuesta1 no)
(respuesta2 no)
(respuesta3 no)
(respuesta4 si)
(respuesta5 no)
(respuesta6 si)
(respuesta7 no)
(respuesta8 no)
(respuesta9 si)
f-10    (respuesta10 no)
f-11    (planta-tallos-debilitados)
f-12    (no-potasio)
f-13    (planta-color-purpura)
f-14    (no-fosforo)
f-15    (raiz-crece-poco)
```

In [ ]: